Video script — MARCO Crystallisation Triage (target: 7:45, hard cap 8:00)

Shooting script for the coursework demo video. Each block has a target duration, what to have on screen, and narration to read/paraphrase in your own words — don't read it robotically, use it as talking points. Numbers below are pulled directly from `docs/report.md`, `experiments/log.csv` and `docs/genai_evaluation.md` — verified against those files, not invented for the video.

Before recording — have these open in tabs/windows ready to switch to, so there's no dead time hunting for files: 1. Terminal, repo root 2. `notebooks/01_eda.ipynb` (rendered, scrolled to the class/source distribution charts) 3. `notebooks/02_baseline.ipynb` (scrolled to the confusion matrix / PR curve) 4. `notebooks/03_ann_initial.ipynb` and `notebooks/06_ann_final.ipynb` (learning curves for both) 5. `docs/report.md`'s "ANN: Initial and Final Models (Q2, Q3)" section (the side-by-side search/learning-curve figures) and "Model Comparison (Q3)" section (the comparison table) 6. Browser at `http://127.0.0.1:8000` (confirm the backend is running — see below) 7. `notebooks/04_xai.ipynb` (Grad-CAM/occlusion/LIME comparison figure) 8. `docs/genai_evaluation.md`'s comparison table 9. The three ready-to-upload misclassified test images in `data/demo_errors/` — the false negative (`01_crystals_false_negative.png`) is the one used live in the demo below

Start the backend before recording:

```
PYTHONPATH=src uvicorn app.backend.main:app --host 127.0.0.1 --port 8000
```

0:00–0:35 — Intro (0:35)

Show: face-cam or title slide with project name.

> *Hi, I'm Chamika Subashinie. This is my MSc Data Science coursework project. I built a model that sorts protein crystallisation drop images into four classes: Clear, Crystals, Precipitate and Other, using the MARCO dataset.*
>
> *The aim is to help a crystallographer decide which drops to look at first. Only drops with crystals are worth harvesting and sending to a synchrotron beamline, so for this user, missing a real crystal is much more costly than a false alarm. That imbalance guided every modelling decision I made.*

0:35–[TBD, ~1:40] — Data download (~1:05) — TIMING NOT YET ADJUSTED, see note at bottom

Show: open `scripts/download_extract.py` in the editor; `notebooks/00_download_explore.ipynb`'s per-shard and pooled chi-square cells; `data/splits.csv` head.

> *The full MARCO dataset is large: about 463,000 images, around 55 gigabytes, stored as TFRecord files split into shards. The training part alone has 407 shards of about 1,000 images each.*
>
> *To understand the data, I first downloaded one shard and counted how many images were in each of the four classes. Then I wanted to know: do all the shards have a similar class mix? So I downloaded two more shards from different parts of the dataset and compared them.*
>
> *The share of Crystals images was close in all three shards: 11.8, 13.2 and 14.8 percent. The full dataset has 12.8 percent. One shard alone can be a little off, but together the three shards matched the full dataset well.*
>
> *This told me that a sample made from several shards would represent the whole dataset, so I used 15 shards, about 14,400 images, for this project. This is the script that downloads these 15 shards and extracts the images.*

EDA — TIMING NOT YET SET (content only, per instruction)

Show: `01_eda.ipynb` — leakage-grouping cells, `scripts/make_splits.py`, the chi-square cell, then the class distribution / source distribution / image-size charts.

> *Next, the exploratory data analysis.*
>
> *Leakage control. The same well is often photographed more than once, and a well can hold several drops. If those images ended up in both training and testing, the test score would look better than it really is. So I read the plate and well numbers from the filenames and gave every image a group key. Images from the same plate and well share a key.*
>
> *Splitting the data. I then split the data into training, validation and test sets, roughly 70, 15 and 15 percent, using scikit-learn's StratifiedGroupKFold. "Group" means whole groups stay together, so no well crosses a split. "Stratified" means each set keeps a similar class balance. I saved the split as a CSV file with each image's ID, label, group key and split. Every model uses this same file, and I can check at any time that no group crosses a split.*

> *Is the sample representative? On the training and validation images, I ran a chi-square test against the class counts of the full MARCO dataset. There was no significant difference, so my sample reflects the population.*
>
> *Exploring the data. I plotted the class distribution, the source institutions, and the image sizes for each class. Image sizes vary widely, which is why every image is resized before training.*

Classical baseline: LightGBM — TIMING NOT YET SET (content only, per instruction)

Show: `02_baseline.ipynb` — confusion matrix and PR curve; optionally the feature-importance chart from `docs/figures/q3_baseline_feature_importance.png`.

> *Classical baseline. Before any neural network, I built a classical model to compare against. Each image is turned into hand-crafted features: HOG, which describes edges and shapes, and LBP, which describes texture. A LightGBM model then learns to classify those features. I chose LightGBM over XGBoost and SVM because it trains quickly on almost 2,000 features and handles class imbalance well. I used balanced class weights, so the rare Crystals class counts as much as the others.*
>
> *Setting the precision rule. On the validation set, I plotted the baseline's precision–recall curve for Crystals. From it I set my rule: at least 30 percent of the drops the model flags must really contain crystals. That means out of every 10 flagged drops, at least 3 are real crystal drops. I fixed this level before training any neural network and used it unchanged for every model, so it can't favour any of them.*
>
> *Result. On the test set, the baseline found 93.7 percent of the crystal drops at this precision. It trained in under 30 seconds, so it's a strong and fast model to beat.*

Initial ANN — TIMING NOT YET SET (content only, per instruction)

Show: `03_ann_initial.ipynb` learning curves and the new failed-experiment figure (`q3_ann_initial_failure.png`); terminal showing `train_ann.py --help` or the hyperparameter search bar chart.

> *Initial ANN. For the neural network, I used a ResNet backbone pretrained on ImageNet. I kept the backbone frozen and trained only a new head on top: a dropout layer and a final layer for my four classes.*
>
> *Hyperparameter search. I searched the three required hyperparameters: learning rate, from 0.0003 to 0.01; batch size, 16, 32 or 64; and network depth, ResNet18 or ResNet34. That's 24 possible combinations. I ran a random search of 12 configurations, each trained for 12 epochs. Because each setting was drawn*

> *independently, one combination came up twice, so the search covered 11 different combinations.*
>
> *Where it ran. Training on my laptop was too slow, so I needed a GPU. I started on Google Colab's free tier, but it kept cutting me off partway through the search. A paid subscription was an option, but I only needed a few hours of GPU time, so renting a GPU on RunPod by the hour was cheaper and more reliable. The training code is exactly the same either way. Only the machine changed.*
>
> *What the learning curves showed. Training and validation loss both dropped in the first epoch, then flattened out, and stayed close together. So the model wasn't overfitting, but it also stopped improving. With the backbone frozen, only about two thousand numbers in the head can learn, and they sit on top of features that were never adapted to drop images. That plateau is what led to my next step: the Final ANN.*
>
> *A failed experiment. One setting failed: the highest learning rate, 0.01, with the smallest batch size, 16. Training became unstable. The loss went up instead of down, and crystal recall dropped to 0.62. I kept it in my log rather than hiding it. It showed that this model needs a smaller learning rate.*

3:20–4:30 — Final ANN: 4th hyperparameter, comparison table, error analysis (1:10)

Show: the side-by-side search/learning-curve figures in `docs/report.md`'s "ANN: Initial and Final Models (Q2, Q3)" section first, then its "Model Comparison (Q3)" section for the table (or the PDF pages).

> *With the lecturer's approval, I added a fourth hyperparameter: degree of fine-tuning — frozen versus unfreezing the backbone's last block, `layer4`, alongside the head. That gives the model real capacity to adapt its highest-level features to crystallisation images specifically, not just reuse ImageNet features as-is.*

> *You can see the effect directly in these two charts — the Final ANN's 12-config search sits uniformly higher than the Initial ANN's, and unlike the Initial ANN, its training loss keeps falling across all 12 epochs instead of plateauing after the first, because there's now real capacity left to learn from.*

> *Here's the full comparison on the held-out test set, each model run exactly once through `final_eval.py`:*
>
> - *Trivial majority-class benchmark: 0.0 recall at the precision floor — obviously unusable, it exists to show accuracy alone is a bad metric: 48% accuracy, zero crystals ever caught.*
> - *Classical baseline: 0.937.*

> - *Initial ANN, frozen: 0.886 — and interestingly, this one actually loses to the baseline on test despite beating it clearly on validation. I report that honestly rather than smoothing it over — it's a genuine validation/test reversal.*
> - *Final ANN, layer4 unfrozen: **0.965** — the best result, and the gap held up on test, which is good evidence the unfreeze produced a real improvement rather than overfitting to validation through repeated model selection.*

> *So my pre-registered expectation — that the ANN would win — was correct, but only once I actually unfroze part of the backbone; the frozen version alone wasn't enough.*

> *I also did a proper error analysis on the Final ANN's test errors, not just the aggregate number. Breaking accuracy down by source institution shows real spread — 60 to 69% on the two smallest, most underrepresented sources, versus up to 85% on the best-performing one — directly confirming the EDA's source-imbalance finding on held-out data, not just a hypothesis. And Precipitate is the single biggest source of confusion for every other class, on both the baseline and the ANN — so that's a dataset-level pattern, not something specific to one architecture.*

4:30–6:10 — Live demo: web app, real-time prediction, Grad-CAM (1:40)

Show: browser at `http://127.0.0.1:8000`. Upload a normal, correctly-classified test image first (e.g. a clear Crystals example).

> *This is the web app — FastAPI backend, simple frontend. Every prediction here runs the actual trained model fresh, right now, nothing is precomputed. I'll upload a crystal image.*

[Upload image, let it run, point at the result]

> *It returns a score for all four classes and the predicted class — and I'm careful about how this is worded: this is the model's raw softmax output, not a calibrated, guaranteed confidence. It reflects patterns learned from training data, not proof of correctness.*

[Point at the Grad-CAM overlay]

> *And this is the explanation feature — Grad-CAM, highlighting the region the model's gradients were most sensitive to for this prediction. I picked Grad-CAM as the primary method because it's fast enough to run live in the app, unlike LIME or occlusion, which I only use for offline comparison. It's presented to a crystallographer as "where to look," not a verdict — it doesn't prove the model's true reasoning, just gives supporting evidence.*

> *I didn't just trust that blindly — I ran two required sanity checks on it. First, occluding the Grad-CAM region on a misclassified image dropped its Crystals probability from 0.624 down to 0.136, confirming the highlighted area is actually driving the prediction, not just correlated with it. Second, re-randomising the trained layers' weights changed the heatmap completely — correlation of minus 0.33 with the original — which confirms the explanation genuinely depends on what the model learned, not just the network's architecture.*

6:10–6:55 — A real incorrect prediction (0:45) — required

Show: upload `data/demo_errors/01_crystals_false_negative.png` — a true Crystals image the Final ANN confidently misclassifies.

> *Now let me show a real failure — this is actually required in my brief, and it's important: no model is perfect, and showing one honestly matters more than pretending it doesn't happen. This image is a true Crystals drop, but the model predicts it with only 0.8% Crystals probability — a confident false negative, which is exactly the costly error type for this project, since a missed crystal could mean losing a hit from scarce protein.*

[Point at Grad-CAM overlay on this image]

> *Looking at the Grad-CAM for this one — in my offline comparison, Grad-CAM, occlusion and LIME all agree and point at the same central region of the drop. So the model isn't looking in a nonsensical place — it's looking at the right area and still getting the label wrong. That tells me this is an error of interpretation, not attention — worth knowing, because it means more data from this kind of drop would help more than architecture changes would.*

6:55–7:35 — GenAI comparison (0:40)

Show: `docs/genai_evaluation.md` comparison table.

> *Finally, I compared all of this against a pretrained vision-language model — Claude Haiku 4.5 — zero-shot, no training on MARCO at all, evaluated on the exact same test images with the same metric. It did **worse than every trained model, and worse than the trivial majority-class benchmark** — 0.0 recall at the precision floor, 26% accuracy. It tends to over-use "Other" as a catch-all and confuses Precipitate with Crystals constantly. I'm reporting that as-is, not softened — it's the honest result.*

> *What it does offer that the trained models don't: zero labelled training data needed, and no training cost — but a real per-prediction cost forever, about 66 cents per thousand images, versus near-zero marginal cost once a trained model exists. And every image sent to it leaves the building, over the network, to a third party — a real*

> *privacy consideration a facility would have to weigh. A realistic use for it isn't replacing the classifier — it's sitting downstream, turning a Grad-CAM-flagged result into a plain-language description for a crystallographer glancing at the output.*

7:35–7:55 — Wrap-up (0:20)

Show: face-cam or the comparison table again.

> *So: a fixed, leakage-checked split, a classical baseline that's genuinely hard to beat, an ANN that only wins once part of the backbone is unfrozen, a sanity-checked Grad-CAM explanation, and a GenAI baseline that shows why domain-specific training still matters. Everything shown today ran live, nothing precomputed. Thanks for watching.*

Timing budget check

STALE — not yet updated for the longer Data download narration below. Data download grew from ~15s to ~1:05 (147+ words). Everything from Data download onward in this table is wrong until the next retime pass; total is currently well over 8:00. Content correctness was prioritised first per instruction — do not record from these timestamps yet.

Section	Duration	Running total
Intro	0:35	0:35
Data download	~1:05 (was 0:15)	~1:40 (stale)
EDA	0:45	1:35
Classical baseline	0:40	2:15
Initial ANN + RunPod	1:05	3:20
Final ANN + comparison + error analysis	1:10	4:30
Live demo (prediction + Grad-CAM + sanity checks)	1:40	6:10
Incorrect prediction (required)	0:45	6:55
GenAI comparison	0:40	7:35
Wrap-up	0:20	7:55

Leaves ~5s buffer under the 8:00 cap. This now explicitly covers all four checklist items: error analysis (Final ANN section), request-time prediction (live demo), a sanity-checked XAI method (both required sanity checks named with their actual numbers), and an incorrect prediction analysed with XAI (required: 1 in video, 3 in the report's "Error Predictions and Analysis (Q3, Q5)" section). If you run long in practice, trim "Data download" or "Wrap-up" first

— not the live demo, the incorrect prediction, or the sanity-check lines, since those are what the checklist is actually looking for.